



Selling Blend tickets inside Whish.

The developer integration guide — list Blend events, price them, take payment on Whish's rail, and let Blend issue the ticket.

HOW IT WORKS

Blend owns the truth. Wish owns the payment.

Every read and write runs through the same engine that powers Blend's own checkout, so whatever an organizer changes on Blend appears in your API instantly. You never reimplement pricing, inventory, or fraud rules — you present them. Wish collects the payment on its own rail; Blend confirms and issues the ticket.

/ 01

Server-authoritative pricing

Your amount must match Blend's total ($\pm \$0.01$) or it's rejected as `AMOUNT_MISMATCH`. Always display the `/quote` numbers.

/ 02

One key, one organizer

A publishable key sees one organizer's opted-in events. Platform-wide coverage = one key per organizer, merged in your backend.

/ 03

Wish is the rail

Create-payment opens a Wish collect session on Blend's account; the buyer settles it natively. Blend confirms only after Wish's status API says success.

/ 04

USD, end to end

The Wish rail is locked to USD across create, confirm and refund. Amounts are major units (dollars), never cents.

The integration flow

1 List events & the event page

`GET /storefront/catalog` → `/events/:id/page` — cards, tickets, waves, add-ons, questions, seating, all live from Blend.

2 Quote the cart

`POST /storefront/quote` returns the server-authoritative total. Display it verbatim.

3 Create the Wish payment

`POST /storefront/wish/create-payment` → `{ url, externalId, registrationId }`.

4 Buyer pays, Blend confirms

The buyer settles inside Wish. Blend re-verifies the collect, claims capacity, books seats, and emails the ticket — server to server.

5 Deliver the ticket

Poll `GET /storefront/registration/:id` until `paid:true`; show the pass and deep-link the QR.

GETTING STARTED

Your first paid ticket

Call the API from your backend. Every request carries your publishable key, an allowlisted `Origin`, and the real buyer IP so Blend's per-buyer rate limits key on the buyer, not your egress.

headers — every request

```
x-blend-key: pk_live_XXXXXXXXXXXXXXXXXXXXXXXXXXXXX
Origin: https://wish.money
Content-Type: application/json
X-Forwarded-For: <the buyer's device IP>
```

the call sequence — paid GA ticket

```
# 1 · browse + render
GET /api/v1/storefront/catalog
GET /api/v1/storefront/events/{id}/page

# 2 · quote (server-authoritative total)
POST /api/v1/storefront/quote
  { "eventId":"aBc123", "selectedTicketId":"665f...", "quantity":2 }

# 3 · open the Wish collect session
POST /api/v1/storefront/wish/create-payment
  Idempotency-Key: <fresh uuid per attempt>
  { "amount":49.5, "currency":"USD", "userName":"Rami K",
    "userEmail":"rami@example.com",
    "tickets":[{"ticketId":"665f...", "quantity":2}] }
  → { url, externalId, registrationId }

# 4 · buyer settles in Wish, then poll
GET /api/v1/storefront/registration/{registrationId}
  → { paid:true, ticket:{ ... } }
```

Idempotency keys must be globally unique

The 15-minute replay cache is keyed on the raw header value. Use a fresh random UUID per checkout attempt and reuse it only to retry the identical request — never derive it from cart contents or reuse across buyers.

Free tickets skip the payment leg: a \$0 order goes to `POST /storefront/register-guest` and confirms instantly with the QR email (send the discount code in the body for the free path to apply).

ENDPOINTS & SEATING

Key endpoints

All paths under `https://api.blendapp.ai/api/v1`. Full field-level contracts live in the reference bundle.

METHOD	PATH	WHAT IT DOES
GET	<code>/storefront/catalog</code>	Event cards for the browse screen
GET	<code>/storefront/events/:id/page</code>	Full event page — tickets, waves, add-ons, questions, seating
POST	<code>/storefront/quote</code>	Server-authoritative total (subtotal + fee)
POST	<code>/storefront/whish/create-payment</code>	Open a Whish collect session; returns the collect URL + ids
POST	<code>/storefront/register-guest</code>	Free / \$0 orders — confirms instantly
GET	<code>/storefront/registration/:id</code>	Order status — poll until <code>paid:true</code>
POST	<code>/storefront/seating/events/:id/hold</code>	Hold reserved seats (10-minute hold)

Reserved seating

When the event page returns `seating.enabled:true`, generate a stable **X-Seat-Identity** per buyer and send it on every seating call *and* on create-payment — the payment re-holds under that identity.

One identity, everywhere

If the identity on the hold and the payment differ, the re-hold fails with `409 SEAT_UNAVAILABLE`. Generate it once per purchase session and reuse it.

HANDLE FAILURE PREDICTABLY

Errors & limits

Errors are `{ success:false, code, message }`. Match on `code || error` — a few endpoints carry the identifier in `error`. Reads/writes rate-limit per buyer; the burst, general and access-code limiters key on your egress IP, so agree an allowlist with Blend before a big on-sale.

CODE	MEANING	DO
<code>AMOUNT_MISMATCH</code>	Amount $\neq$ server total	Re-quote, re-present, retry with a new key
<code>ORIGIN_NOT_ALLOWED</code>	Origin missing / not allowlisted	Send an allowlisted <code>Origin</code> header
<code>SOLD_OUT</code>	Tier or event sold out	Remove from sale, offer waitlist
<code>waiting_room_required</code>	High-demand queue active	Run the waiting-room flow first
<code>SEAT_UNAVAILABLE</code>	Seat lost or identity mismatch	Refresh availability, re-select
<code>EVENT_CANCELLED</code>	Event cancelled	Block purchase, pull from catalog

Go-live checklist

- Provision your key(s); allowlist your backend `Origin` (s) per organizer.
- Agree an egress-IP allowlist / limiter tier with Blend before any on-sale.
- Drive the flow from the event-page payload — questions, add-ons, seating appear only when the event has them.
- Quote on every cart change; send a fresh UUID `Idempotency-Key` per attempt.
- Poll `registration/:id` until `paid:true`; deep-link the ticket page for the QR.
- Run the full test matrix on staging before shipping.

Full reference

Every field, limit and error code is in the developer bundle. Integration support: developers@blendapp.ai — include the `registrationId` / `externalId` with any payment question.